

Report Requirements

1 Design and Implementation

Describe your design decisions and implementation details:

- Design overview.

这是一个基于 Tomasulo 算法的推测执行（Speculative Execution）RISC-V 模拟器的设计概览。该模拟器通过实现乱序执行和顺序提交，旨在模拟现代高性能处理器的流水线行为。

我的整个设计是通过增加InstEntry和DecodeInstEntry,ExecuteEntry这两个接口和原本riscv.cc中的机制相融合

```
struct InstEntry {
    uint32_t pc;
    uint32_t raw_inst;
    std::string inst_str;

    RISC_V::InstType op_type;
    RISC_V::RegId dest_reg;
    RISC_V::RegId rs1_reg;
    RISC_V::RegId rs2_reg;
    int64_t immediate;

    // Branch Prediction Result (Task 4)
    bool pred_taken = false;
    uint32_t pred_target_pc = 0;
    bool has_exception = false;
    enum class Exception { NO=0, ILLEGAL_INSTRUCTION=2 };
    uint32_t exception_cause = 0; // 例如: ILLEGAL_INSTRUCTION = 2
};
```

```

void TomasuloSimulator::DecodeInstEntry(InstEntry &inst, const Regs &regs) {
    PipeOp op{};
    op.pc = inst.pc;
    op.inst = inst.raw_inst;
    if(enable_predict) op.has_exception=true;
    DecodeInst(&op, regs);

    inst.inst_str = op.inst_str;
    inst.op_type = op.inst_type;
    inst.dest_reg = op.dest_reg;
    inst.rs1_reg = op.rs1;
    inst.rs2_reg = op.rs2;
    inst.immediate = op.offset;
    inst.pred_target_pc = op.branch_target;
    inst.has_exception= op.is_illegal;
    inst.exception_cause= op.is_illegal ? static_cast<uint32_t>(InstEntry::Exception::ILLEGAL_INST) : 0;
    if (op.inst_type == ECALL) {
        inst.rs1_reg = REG_A0;
        inst.rs2_reg = REG_A7;
        inst.dest_reg = REG_A0;
    }
}

```

```

void TomasuloSimulator::ExecuteEntry(RsEntry &rs, MemoryManager *mem) {
    PipeOp op{};
    int rob_id = rs.dest_rob_id;
    op.pc = rob_[rob_id].pc;
    op.inst = rob_[rob_id].inst;
    op.inst_type = rob_[rob_id].op_type;
    if (RISCV::JType(op.inst_type)) {
        op.op1 = rs.a;
    } else {
        op.op1 = rs.vj;
    }
    // op2需要特殊处理
    if (RISCV::IType(op.inst_type)) {
        op.op2 = rs.a;
    } else {
        op.op2 = rs.vk;
    }
    op.offset = rs.a;
    // if (exit_ctrl) {
    //     printf("Program exit from an exit() system call\n");
    //     if (dump_history_) {
    //         printf("Dumping history to dump.txt...");
    //         DumpHistory();
    //     }
    //     PrintStatistics();
    //     exit(0);
    // }
    try {
        ExecuteInst(&op, &rob_[rob_id].exit_ctrl, mem);
    } catch (const std::runtime_error& e) {
        // 专门捕获 runtime_error, 包括您的 "Unknown syscall type"
        op.ex_illegal=true;
        rob_[rob_id].has_exception=true;
        rob_[rob_id].exception_cause=static_cast<uint32_t>(RobEntry::Exception::ILLEGAL_EXE);
    }
    if (IsReadMem(op.inst_type)) {
        rs.a = op.out;
        rob_[rob_id].read_mem = op.read_mem;
        rob_[rob_id].read_sign_ext = op.read_sign_ext;
        rob_[rob_id].mem_len = op.mem_len;
        rob_[rob_id].mem_address = op.out;
    } else if (IsWriteMem(op.inst_type)) {
        rs.a = op.out;
        rob_[rob_id].write_mem = op.write_mem;
    }
}

```

```

    rob_[rob_id].mem_len = op.mem_len;
    rob_[rob_id].mem_address = op.out;
    rob_[rob_id].value = op.op2;
} else if (IsBranch(op.inst_type)) {
    rob_[rob_id].branch = op.branch;
    rob_[rob_id].jump_pc = op.jump_pc;
    // printf("jmp %lx-pc:%lx\n", rob_[rob_id].jump_pc, rob_[rob_id].pc);
} else if (IsJump(op.inst_type)) {
    rob_[rob_id].branch = op.branch;
    rob_[rob_id].jump_pc = op.jump_pc;
    rob_[rob_id].value = op.out;
    // printf("jmp %lx-pc:%lx\n", rob_[rob_id].jump_pc, rob_[rob_id].pc);
} else {
    rob_[rob_id].value = op.out;
}
}
}

```

整体设计概览 (Design Overview)

该模拟器严格遵循经典的 Tomasulo 结构，采用一个五级流水线（Fetch, Issue, Execute, Write Result, Commit），并围绕四个核心数据结构运行，以实现乱序执行、数据旁路（Data Forwarding）和分支推测。

A. Fetch (取指)

功能： 从内存中读取指令并推送到指令队列 (`inst_queue_`)。

- **推测处理：** 如果指令是分支，调用分支预测器 (`predictor_`) 预测分支方向和目标 PC。
- **PC 更新：** 根据预测结果 (`inst.pred_taken`) 更新下一周期的 PC。
- **阻塞：** 如果指令队列 (`inst_queue_`) 已满，取指阻塞。

B. Issue (发射 / 重命名)

功能： 将指令从 IQ 分配到 RS 和 ROB，执行寄存器重命名。

- **资源分配：** 检查并分配空闲的 ROB 条目和对应类型的 RS 条目。若任一资源不足，则阻塞。
- **操作数获取 (Renaming)：**
 - 检查 RST：如果源寄存器被 ROB 标签标记 (`rst_[reg].busy`)，则依赖于该 ROB 结果。
 - 若 ROB 结果已就绪 (`rob[h].ready`)，则获取结果值 (`vj / vk`)。
 - 若 ROB 结果未就绪，则记录 ROB 标签 (`qj / qk`) 进行等待。
 - 否则，从寄存器文件 (`regs_`) 获取操作数。
- **RST 更新：** 如果指令有目标寄存器 (`dest_reg != 0`)，更新 RST，将其标记为忙，并记录新的 ROB ID。

C. Execute (执行)

功能： 检查 RS 依赖，在操作数就绪后开始执行，并模拟功能单元延迟。

- **执行开始：** 仅当 `rs.qj == -1` 且 `rs.qk == -1` 时（所有依赖都已解决）才允许执行。
- **延迟模拟：** 对于多周期指令（如乘法），通过 `remaining_cycles` 计时器模拟延迟。
- **Load/Store 地址计算：** 内存指令在此阶段计算出有效的内存地址。
- **内存顺序检查 (Load)：** Load 指令在 Execute 阶段的最后（准备 CDB 之前）检查其之前的 Store 指令，防止地址冲突或 Store 地址未计算完成时进行乱序 Load。
- **结果准备：** 计算结果（包括地址、分支结果、运算结果）被写入对应的 ROB 条目。

D. WriteResult (写回 / CDB 广播)

功能： 将执行结果通过公共数据总线 (CDB) 广播给所有等待的单元。

- **ROB 标记：** 目标 ROB 条目被标记为 `ready = true`。
- **数据旁路：** 遍历所有 RS，清除依赖于此 ROB ID 的 `qj / qk` 标签，并用广播的结果值更新 `vj / vk`。
- **分支检查：** 在此阶段，分支指令会检查实际执行结果和预测结果是否一致，若不一致，则设置 `rob[b].mispredicted = true`。
- **RS 释放：** 执行完成后，对应的 RS 条目被立即释放。

E. Commit (提交)

功能： 顺序提交 ROB 头部已就绪的指令，将推测结果写入体系结构状态。

- **顺序保证：** 仅检查 ROB 头部指令。如果 `!head.ready`，则提交阻塞。
- **状态更新：**
- **Store：** 在此阶段执行实际的**内存写入**，确保 Store 的顺序可见性。
- **写寄存器指令 (ALU/Load)：** 将结果写入**寄存器文件** (`regs_`)。
- **RST 清除：** 如果提交的指令是目标寄存器的最后一条写指令，清除 RST 上的 ROB 标签。
- **ROB 释放：** 释放 ROB 头部条目，推进 `rob_head_`。
- **错误恢复：** 如果 ROB 头部的指令是**错误预测的分支** (`head.mispredicted = true`)，则调用 `FlushPipeline` 清空整个流水线，并从正确的目标 PC 重新开始取指。

- Data structure designs for ROB, RS, RST.

```

struct RstEntry {
    int reorder_rob_id = -1;
    bool busy = false;
};

```

```

struct RobEntry {
    bool busy = false;
    bool ready = false;
    enum class State { UNKNOWN, Issue, Execute, Write, Commit, EXCPT };
    State state = State::UNKNOWN;
    uint64_t pc = 0;
    uint32_t inst = 0;
    std::string inst_str;
    RISCVC::InstType op_type;
    bool has_exception = false;
    enum class Exception { NO=1, ILLEGAL_INSTRUCTION=2, ILLEGAL_EXE=3 };
    uint32_t exception_cause = 0;
    RISCVC::RegId dest_reg = 0;
    uint64_t mem_address = 0;
    uint64_t value = 0;
    bool mispredicted = false;
    bool branch = false;
    uint32_t jump_pc = 0;
    bool write_mem = false;
    bool read_mem = false;
    bool read_sign_ext = false;
    uint32_t mem_len = 0;
    bool exit_ctrl = false;
    bool pred_taken = false;
    uint32_t pred_target_pc = 0;
};

```

```

struct RsEntry {
    bool busy = false;
    RISCVC::InstType op_type;
    int dest_rob_id = -1;
    uint64_t vj = 0, vk = 0;
    int qj = -2, //-2:没有使用 -1: 没有
        qk = -2;
    int64_t a = 0;
    int remaining_cycles = 0;
};

```

```

struct InstEntry {
    uint32_t pc;
    uint32_t raw_inst;
    std::string inst_str;

    RISCVC::InstType op_type;
    RISCVC::RegId dest_reg;
    RISCVC::RegId rs1_reg;
    RISCVC::RegId rs2_reg;
    int64_t immediate;

    // Branch Prediction Result (Task 4)
    bool pred_taken = false;
    uint32_t pred_target_pc = 0;
    bool has_exception = false;
    enum class Exception { NO=0,ILLEGAL_INSTRUCTION=2};
    uint32_t exception_cause = 0; // 例如: ILLEGAL_INSTRUCTION = 2
};

```

```

struct CdbResult {
    bool valid = false;
    int rob_id = -1;
};

```

- Detailed explanation of your implementation of each pipeline stage (Issue, Execute, Write Result, Commit).

阶段 1: Issue (发射 / 寄存器重命名)

`TomasuloSimulator::Issue()` 负责从指令队列 (`inst_queue_`) 取出指令, 并将其分配给 ROB 和 RS。

1. 资源检查与分配:

- 首先通过 `FindFreeRS()` 查找对应功能单元 (ALU, Mem, Mul) 的空闲保留站。
- 接着调用 `AllocateRobEntry()` 查找空闲的 ROB 条目。
- **阻塞条件:** 如果 RS 或 ROB 满 (返回 -1), 则 `Issue()` 阶段立即停止, 从而阻塞取指 (Fetch) 和后续指令的发射。

2. ROB/RS 初始化:

- 分配成功后, 指令的详细信息 (PC、指令码、目标寄存器、指令字符串等) 被复制到新分配的 **ROB 条目**。
- 新分配的 **RS 条目** 被初始化, 设置 `busy = true`, 并记录目标 ROB ID (`dest_rob_id`) 和立即数 (`rs.a`)。

3. 操作数获取与寄存器重命名 (Reg Renaming):

- 使用 `fetch_operand Lambda` 函数处理指令的源操作数 (rs1/rs2)。
- **检查 Register Status Table (RST):**
- 如果 `rst_[reg_id].busy` 为真且 `reorder_rob_id` 有效 (表示结果尚未计算), 则检查该 ROB 条目是否 `ready` :
- **Ready:** 从 ROB 中直接获取值 (`v_field = rob_[h].value`), 并设置 `q_field = -1` (值可用)。
- **Pending:** 记录 ROB 标签 (`q_field = h`), 表示需要等待 ROB `h` 的结果。
- 如果 RST 不忙, 则操作数从 **寄存器文件** (`regs_[reg_id]`) 中获取。

4. RST 更新:

- 如果指令会写回目标寄存器 (`inst.dest_reg != 0`) 且不是分支/Store 指令, 则将 RST 对应条目设置为 `busy = true`, 并用新的 ROB ID 进行**标记** (`rst_[inst.dest_reg].reorder_rob_id = rob_id`), 实现寄存器重命名。

阶段 2: Execute (执行)

`TomasuloSimulator::Execute()` 负责检查保留站的依赖性, 并在满足条件时开始或推进指令执行。

1. 依赖性检查:

- 迭代 ALU, Mem, Mul 三种保留站。
- 指令执行的前提是: `rs.qj == -1` 且 `rs.qk == -1` (所有源操作数都已就绪)。

2. 延迟模拟:

- 如果启用了 `enable_delay`，则使用 `rs.remaining_cycles` 模拟功能单元延迟。每周期递减 1，直到剩余 1 周期时才调用 `ExecuteEntry`。
- （代码中为乘法指令设置了 5 周期延迟）。

3. 执行核心 (`ExecuteEntry`):

- 调用 `ExecuteEntry` 实际执行指令，计算结果，并更新相应的 ROB 条目。
- **分支/跳转：** 更新 `rob_[rob_id].jump_pc` 和 `rob_[rob_id].branch`（实际是否跳转）。
- **内存操作：** 对于 Load/Store，计算出内存地址，并更新 `rob_[rob_id].mem_address` 和 `mem_len`。Store 的值也被存入 `rob_[rob_id].value`。
- **异常处理：** 如果在执行中捕获到 `runtime_error`（例如非法系统调用），ROB 状态会被设置为 `EXCPT`。

4. Load/Store 内存顺序检查 (在 `CDB_rs Lambda` 中):

- 对于 **Load** 指令 (`rob_[rob_id].read_mem`)，在实际访问内存之前，会检查 ROB 中所有**更早发射但尚未提交的 Store** 指令。
- 如果存在一个先行 Store 尚未计算出地址 (`rob_[i].state==RobEntry::State::Issue`) 或地址相同，Load 就会被 **Stall (continue)**，以保证 Load 在 Store 之后顺序访问内存。

5. RS 释放与 CDB 准备:

- 一旦执行和内存操作 (Load) 完成，RS 立即被释放 (`rs.busy = false`)。
- 结果被打包成 `CdbResult` 推入 `cdb_results_` 队列。

阶段 3: WriteResult (写回 / CDB 广播)

`TomasuloSimulator::WriteResult()` 负责将执行结果广播给所有等待的单元。

1. ROB 状态更新:

- 遍历 `cdb_results_` 队列，找到对应的 ROB 条目 `b`。
- 将 `rob_[b].ready` 设为 `true`，状态设为 `Write`。

2. 分支预测结果检查与更新:

- 对于分支指令，将执行结果 (`rob_[b].jump_pc`) 与预测目标 (`rob_[b].pred_target_pc`) 进行比较。
- 如果两者不匹配，则设置 `rob_[b].mispredicted = true`。
- 如果启用了 `enable_predict`，还会调用 `predictor_->UpdateStrategy` 更新分支预测器状态，并增加 `history_.correct_branch_count` 或设置 `mispredicted` 标志。

3. CDB 广播:

- 遍历所有 RS 条目 (rs_alu_ , rs_mem_ , rs_mul_)。
- 如果某个 RS 条目正在等待当前 ROB 结果 (rs.qj == b 或 rs.qk == b)，则将结果值 (rob_[b].value) 写入该 RS 的 vj / vk 字段，并清除依赖标签 (qj / qk = -1)。

阶段 4: Commit (提交)

TomasuloSimulator::Commit() 严格按程序顺序将结果写入最终的体系结构状态。

1. 顺序与就绪检查：

- 只检查 ROB 头部 (rob_head_) 的指令。
- 如果 head.ready 为 false ，提交阻塞 (return)。

2. Store/内存提交：

- 如果指令是 Store (head.write_mem)，则在此阶段执行实际的**内存写入** (memory_->SetXXX)。这确保了内存操作是**顺序提交**的。

3. 寄存器写回：

- 如果指令写回寄存器 (W2Rd(head.op_type) ，如 ALU/Load)，则将结果 (head.value) 写入**寄存器文件** (regs_[d])。

4. RST 清除：

- 如果当前提交的指令是**最后一条**（即其 ROB ID 仍留在 RST 中）写目标寄存器的指令 (rst_[d].reorder_rob_id == rob_head_)，则清除 RST 条目 (rst_[d].busy = false)。

5. 分支错误恢复：

- 如果 ROB 头部的分支/跳转指令 head.mispredicted 为 true ，则调用 FlushPipeline(head.jump_pc) 进行恢复。

6. ROB 释放：

- 无论是哪种指令，最后都会释放 ROB 头部条目 (head.busy = false)，并推进 rob_head_ 。

• How you handle edge cases:

- ROB full condition.
- Reservation station full condition (for specific types).
- Branch misprediction recovery.
- Memory ordering enforcement.

1 ROB 满条件处理

实现：在 `TomasuloSimulator::AllocateRobEntry()` 中通过 `IsRobFull()` 检查 `rob_count_ >= ROB_SIZE`。

- **影响：**如果 ROB 满，`AllocateRobEntry()` 返回 `-1`。这会导致 `Issue()` 阶段立即停止处理指令队列中的指令，从而**阻塞整个流水线**（包括 `Fetch` 阶段）。

2 Reservation Station 满条件处理

实现：在 `TomasuloSimulator::FindFreeRS(type)` 中，它遍历对应功能单元的 RS 数组（`rs_alu_`，`rs_mem_`，`rs_mul_`）查找 `!busy` 的条目。

- **影响：**如果 `FindFreeRS()` 返回 `-1`，表示 RS 满。这会导致 `Issue()` 阶段立即停止，**阻塞流水线**。

3 分支预测错误恢复 (Branch Misprediction Recovery)

我的恢复机制遵循严格的 **Commit 阶段恢复**：

1. **错误检测 (WriteResult)：**在 `WriteResult()` 阶段，当分支指令执行完毕并准备写回时，会检查实际跳转目标（`rob[b].jump_pc`）是否与预测目标（`rob[b].pred_target_pc`）一致。如果不一致，则设置 `rob[b].mispredicted = true`。
2. **错误确认与恢复 (Commit)：**

- 只有当带有 `mispredicted = true` 标志的分支指令到达 **ROB 头部**（`rob_head_`）时，错误才会被正式确认。
- 调用 `FlushPipeline(head.jump_pc)`，传入正确的下一 PC。

3. Flush 动作 (FlushPipeline)：

- **PC 恢复：**`pc_ = correct_pc`（设置为正确的跳转地址）。
- **ROB 清理：**`rob_tail_` 被设置为 `rob_head_ + 1`，并将所有除 `rob_head_` 之外的 ROB 条目清除/重置。
- **RS 清理：**所有保留站（`rs_alu_`，`rs_mem_`，`rs_mul_`）被完全清除（`std::fill`）。
- **RST 清理：**Register Status Table 被完全清除。
- **指令队列清理：**`inst_queue_` 被清除。

4 内存顺序强制 (Memory Ordering Enforcement)

`Store` 和 `Load` 指令在 Tomasulo 结构中的顺序执行是通过 ROB 和 RS 间的特殊逻辑强制执行的。

- **Store 提交顺序：**

- Store 指令在 **Execute** 阶段计算出地址 (rob_[rob_id].mem_address) 和要存储的值 (rob_[rob_id].value)。
- **实际内存写入**操作 (memory_->SetXXX) 被严格推迟到 **Commit** 阶段，并且只发生在它到达 ROB 头部时。这保证了 Store 指令的内存写入是**严格按程序顺序**进行的。
- **Load 转发与乱序执行：**
- **RAW 依赖 (Load vs. Store)：** 在 **Execute** 阶段的 CDB_rs Lambda 中，对于 Load 指令，它会检查所有在 ROB 中**更早发射且未提交的 Store**。
- 如果发现先行 Store 尚未计算地址 (状态仍为 Issue)，Load 必须等待。
- 如果发现先行 Store 地址相同，Load 必须等待，直到 Store 提交其值或地址冲突解决。
- 我的代码实现了这种检查：

// 在它之前发射但尚未提交

```
for(int i=rob_head_;i!=rob_id;i=(i+1)%ROB_SIZE){
    if(rob_[i].busy && IsWriteMem(rob_[i].op_type) &&
        (rob_[i].state==RobEntry::State::Issue || (rob_[i].mem_address==rob_[rob_id].mem_address))) {
        flag=true; // 发现冲突或先行Store地址未计算
    }
}
if(flag) continue; // Stall the load
```

2 Correctness Verification

Document your testing and verification approach:

- Test cases used and expected vs. actual results.

```
xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_arithmetic.riscv --pipeline_mode "speculative Tomasulo"
30
-10
370350
411
49380
771
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 725
Number of Cycles: 1155
Avg Cycles per Instruction: 1.5931
Branch Prediction Accuracy: 0.00% (0 / 0)
-----
xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_arithmetic.riscv
30
-10
370350
411
49380
771
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 726
Number of Cycles: 1598
Avg Cycles per Instruction: 2.2011
Number of Control Hazards: 296
Number of Data Hazards: 575
-----
```

- Evidence of correct branch misprediction recovery (e.g., by showing that the final architectural state matches that of the five-stage reference simulator).

```

xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_rem.riscv --pipeline_mode "speculative Tomasulo" -p --branch_predict_policy "One-bit Predictor (1-Bit)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 661
Number of Cycles: 770
Avg Cycles per Instruction: 1.1649
Branch Prediction Accuracy: 64.41% (114 / 177)
xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_rem.riscv
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 662
Number of Cycles: 1459
Avg Cycles per Instruction: 2.2039
Number of Control Hazards: 284
Number of Data Hazards: 512
xzx@xzx-virtual-machine:~/CAII/Lab5$

```

3 Performance Analysis

Present your performance evaluation results:

- Performance comparison tables: cycle count, IPC for each test program (Tomasulo vs. five-stage), hazard count (for the speculative Tomasulo simulator, structural hazards should be included).

1. speculative Tomasulo simulator without predictor

```

xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/ackermann.riscv --pipeline_mode "speculative Tomasulo" -p
Ackermann(0,0) = 1
Ackermann(0,1) = 2
Ackermann(0,2) = 3
Ackermann(0,3) = 4
Ackermann(0,4) = 5
Ackermann(1,0) = 2
Ackermann(1,1) = 3
Ackermann(1,2) = 4
Ackermann(1,3) = 5
Ackermann(1,4) = 6
Ackermann(2,0) = 3
Ackermann(2,1) = 5
Ackermann(2,2) = 7
Ackermann(2,3) = 9
Ackermann(2,4) = 11
Ackermann(3,0) = 5
Ackermann(3,1) = 13
Ackermann(3,2) = 29
Ackermann(3,3) = 61
Ackermann(3,4) = 125
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431308
Number of Cycles: 575651
Number of Control Hazards: 48105
Avg Cycles per Instruction: 1.3347
Branch Prediction Accuracy: 8.08% (13852 / 171540)

```

2. speculative Tomasulo simulator with predictor

```

xzx@xzx-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/ackermann.riscv --pipeline_mode "speculative Tomasulo" -p --branch_predict_policy "One-bit Predictor (1-Bit)"
Ackermann(0,0) = 1
Ackermann(0,1) = 2
Ackermann(0,2) = 3
Ackermann(0,3) = 4
Ackermann(0,4) = 5
Ackermann(1,0) = 2
Ackermann(1,1) = 3
Ackermann(1,2) = 4
Ackermann(1,3) = 5
Ackermann(1,4) = 6
Ackermann(2,0) = 3
Ackermann(2,1) = 5
Ackermann(2,2) = 7
Ackermann(2,3) = 9
Ackermann(2,4) = 11
Ackermann(3,0) = 5
Ackermann(3,1) = 13
Ackermann(3,2) = 29
Ackermann(3,3) = 61
Ackermann(3,4) = 125
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431308
Number of Cycles: 534113
Number of Control Hazards: 34259
Avg Cycles per Instruction: 1.2384
Branch Prediction Accuracy: 27.93% (34387 / 123131)

```

3. five-stage simulator

```
xzx@xzx-virtual-machine:~/CAII/lab5$ ./build/simulator -i test/build/with-syscall/ackermann.riscv
```

```
Ackermann(0,0) = 1
Ackermann(0,1) = 2
Ackermann(0,2) = 3
Ackermann(0,3) = 4
Ackermann(0,4) = 5
Ackermann(1,0) = 2
Ackermann(1,1) = 3
Ackermann(1,2) = 4
Ackermann(1,3) = 5
Ackermann(1,4) = 6
Ackermann(2,0) = 3
Ackermann(2,1) = 5
Ackermann(2,2) = 7
Ackermann(2,3) = 9
Ackermann(2,4) = 11
Ackermann(3,0) = 5
Ackermann(3,1) = 13
Ackermann(3,2) = 29
Ackermann(3,3) = 61
Ackermann(3,4) = 125
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 431309
Number of Cycles: 1006249
Avg Cycles per Instruction: 2.3330
Number of Control Hazards: 123904
Number of Data Hazards: 451035
-----
```

- Impact of M-extension latency on performance (with and without 4-cycle delay).

-e option is used to control M-extension latency

```
xzx@xzx-virtual-machine:~/CAII/lab5$ ./build/simulator -i test/build/with-syscall/test_rem.riscv -e --pipeline_mode "speculative Tomasulo" -p --branch_predict_policy "One-bit Predictor (1-Bit)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 661
Number of Cycles: 782
Avg Cycles per Instruction: 1.1831
Branch Prediction Accuracy: 64.41% (114 / 177)
-----
```

```
xzx@xzx-virtual-machine:~/CAII/lab5$ ./build/simulator -i test/build/with-syscall/test_rem.riscv --pipeline_mode "speculative Tomasulo" -p --branch_predict_policy "One-bit Predictor (1-Bit)"
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 661
Number of Cycles: 770
Avg Cycles per Instruction: 1.1649
Branch Prediction Accuracy: 64.41% (114 / 177)
-----
```

- Impact of microarchitectural parameters: ROB size, number of reservation stations per type.

```
xzx@xzx-virtual-machine:~/CAII/lab5$ ./build/simulator -i test/build/with-syscall/matrixmulti.riscv --pipeline_mode "speculative Tomasulo" -p --branch_predict_policy "One-bit Predictor (1-Bit)" --RS_ALU_SIZE 10 --RS_MEM_SIZE 10 --RS_MUL_SIZE 10
The content of A is:
0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9
The content of B is:
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
The content of C=A*B is:
0 0 0 0 0 0 0 0 0 0
0 10 20 30 40 50 60 70 80 90
0 20 40 60 80 100 120 140 160 180
0 30 60 90 120 150 180 210 240 270
0 40 80 120 160 200 240 280 320 360
0 50 100 150 200 250 300 350 400 450
0 60 120 180 240 300 360 420 480 540
0 70 140 210 280 350 420 490 560 630
0 80 160 240 320 400 480 560 640 720
0 90 180 270 360 450 540 630 720 810
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 75330
Number of Cycles: 79267
Number of Control Hazards: 938
Avg Cycles per Instruction: 1.0523
Branch Prediction Accuracy: 80.18% (2318 / 2891)
-----
```

```

xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/matrixmulti.riscv --pipeline_mode "speculative tomasulo" -p --branch_predict_policy "One-bit Predictor (1-B
it)" --RS_ALU_SIZE 10 --RS_MEM_SIZE 10 --RS_MUL_SIZE 10 --IQ 40
The content of A is:
0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9
The content of B is:
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
The content of C is:
0 0 0 0 0 0 0 0 0 0
0 10 20 30 40 50 60 70 80 90
0 20 40 60 80 100 120 140 160 180
0 30 60 90 120 150 180 210 240 270
0 40 80 120 160 200 240 280 320 360
0 50 100 150 200 250 300 350 400 450
0 60 120 180 240 300 360 420 480 540
0 70 140 210 280 350 420 490 560 630
0 80 160 240 320 400 480 560 640 720
0 90 180 270 360 450 540 630 720 810
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 75330
Number of Cycles: 79267
Number of Control Hazards: 938
Avg Cycles per Instruction: 1.0523
Branch Prediction Accuracy: 72.98% (2318 / 3176)
-----

```

- Branch prediction impact: misprediction rate and performance correlation.

```

xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_branch.riscv --pipeline_mode "speculative Tomasulo" -e -p --branch_predict_policy "One-bit Predictor (
1-Bit)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1158
Number of Cycles: 1344
Number of Control Hazards: 56
Avg Cycles per Instruction: 1.1606
Branch Prediction Accuracy: 66.67% (146 / 219)
-----
xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_branch.riscv --pipeline_mode "speculative Tomasulo" -e -p --branch_predict_policy "Two-bit Predictor (
2-Bit)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1158
Number of Cycles: 1332
Number of Control Hazards: 52
Avg Cycles per Instruction: 1.1503
Branch Prediction Accuracy: 73.17% (150 / 205)
-----
xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_branch.riscv --pipeline_mode "speculative Tomasulo" -e -p --branch_predict_policy "Perceptron Predicto
r"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1158
Number of Cycles: 1368
Number of Control Hazards: 64
Avg Cycles per Instruction: 1.1813
Branch Prediction Accuracy: 59.32% (140 / 236)
-----

```

```

xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_branch.riscv --pipeline_mode "speculative Tomasulo" -e -p --branch_predict_policy "Always Not Taken (N
T)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1158
Number of Cycles: 1728
Number of Control Hazards: 184
Avg Cycles per Instruction: 1.4922
Branch Prediction Accuracy: 3.92% (17 / 434)
-----
xz@xz-virtual-machine:~/CAII/Lab5$ ./build/simulator -i test/build/with-syscall/test_branch.riscv --pipeline_mode "speculative Tomasulo" -e -p --branch_predict_policy "Always Taken (AT)"
Yes, f2 is true
a[5] = 1 2 3 4 5
a[5] = 1 12 123 1234 12345
Program exit from an exit() system call
----- STATISTICS -----
Number of Instructions: 1158
Number of Cycles: 1317
Number of Control Hazards: 47
Avg Cycles per Instruction: 1.1373
Branch Prediction Accuracy: 78.97% (154 / 195)
-----

```

4 (Optional) Discussion and Conclusions

Reflect on your implementation experience:

- Challenges encountered and how you resolved them.
1. 使用乱序执行的分支预测机制时由于可能会取到非指令的数据，因此需要加入异常处理机制，同时还要对取指和发射的部分做出处理
 2. 使用乱序执行的分支预测机制在分支预测错误执行的过程中对于ECALL指令引起的未知类型异常要在执行阶段进行捕获，等待后续执行流刷新rob，rst
 3. 对于ECALL指令中结束程序运行的判断要放在Commit阶段

- Any optimizations you made.

融合了多种分支预测机制